새싹 학습 노트 · 2026.10.30 · Day10 · Optimizer와 Regularization · 수강생용 개념 지도

학습을 안정시키는 도구들은 어디에 있을까

개념을 외우기 전에, **학습 코드의 어느 자리에서 무엇을 바꾸는 도구인지** 지도로 먼저 보세요.

CONCEPT MAP

개념 지도 · 학습 한 번의 안쪽으로

강의 PDF p.3의 겹친 상자를 학습 코드 구조로 펼쳤습니다. 바깥 상자일수록 드물게, 안쪽 상자일수록 자주 실행됩니다. 각 칩은 **그 자리에서 무엇을 하는지**와 **어떤 질문(①~④)에 답하는지**를 보여 줍니다.

실험 한 번 `조건 하나마다 × 1`

여기서 바꾸는 것: **어떤 데이터로 배우고 고를지, 무엇을 비교할지**

시작 전 · 준비

train / valid / test 분리
train은 배우기, valid는 고르기, test는 끝까지 보관
④ 측정·선택

Seed · 재현 조건
`torch.manual_seed()` · 버전·device·설정 기록
④ 측정·선택

바꿀 조건 하나 정하기
기준선 대비 한 가지만 바꿔야 원인을 말할 수 있음
④ 측정·선택

Epoch 반복 `× epochs`

여기서 바꾸는 것: **얼마나 오래 배울지, 학습률을 언제 줄일지, 언제 멈출지**

Batch 한 번 = Iteration 1회 `× ⌈train 개수 ÷ batch size⌉`

여기서 바꾸는 것: **기울기를 몇 개 샘플로 추정할지, 어떤 규칙으로, 얼마나 크게** 가중치를 바꿀지

1 Batch 꺼내기 → `for x, y in loader`

Batch size
작을수록 기울기 잡음↑ 메모리↓
②③ 최적화·일반화

shuffle · drop_last
순서 섞기 · 마지막 작은 batch 처리
② 최적화

2 기울기 비우기 → `zero_grad()`

zero_grad
이전 batch의 기울기 제거
② 최적화

3 예측 (forward) → `pred = model(x)`

모델 내부 · 층마다 반복

`Linear` → **BatchNorm** 중간값을 batch 통계로 표준화 ②③ 최적화·일반화 → `ReLU` → **Dropout** 일부 중간값을 무작위로 0 ③ 일반화

train 모드: Dropout 켜짐 · BatchNorm은 현재 batch 통계
eval 모드: Dropout 꺼짐 · BatchNorm은 저장된 running 통계

4 오차 계산 → `loss_fn(pred, y)`

Loss
줄일 대상 숫자 하나
① 손실

5 기울기 계산 → `loss.backward()`

Gradient
loss가 커지는 방향
② 최적화

6 가중치 갱신 → `optimizer.step()`

Optimizer
어떤 규칙으로? SGD · Momentum · AdamW
② 최적화

Learning rate
얼마나 크게? 갱신 한 번의 보폭
②③ 최적화·일반화

Weight Decay
갱신 때 가중치를 조금씩 줄임
③ 일반화

Epoch 끝 · train 전체를 한 바퀴 돈 뒤

1 평가 모드로 → `model.eval()` `no_grad()`

train() / eval() 모드
Dropout·BatchNorm 동작 전환
④ 측정·선택

2 valid 평가 · 기록 → `history.append(...)`

학습곡선
train·valid loss를 epoch마다 기록
④ 측정·선택

3 학습률 조정 → `scheduler.step(valid_loss)`

Scheduler
valid가 정체되면 lr 감소
②③ 최적화·일반화

4 멈출지 판단 → `patience · min_delta`

Early Stopping
best 갱신 시 저장, 개선 없으면 중단
③④ 일반화·선택

5 다음 epoch로 → `model.train()`

Epoch 수
길수록 더 맞추지만 과적합 위험
②③ 최적화·일반화

학습이 끝난 뒤 · 정리

1 best 상태 복원 → `load_state_dict(best_state)`

best ≠ 마지막
멈춘 시점이 아니라 valid가 가장 좋았던 시점
④ 측정·선택

2 실험표에 기록 → `history CSV · 비교표`

실험표
바꾼 조건 · best epoch · best valid loss · 근거
④ 측정·선택

3 test는 마지막에 한 번 → 모든 선택이 끝난 뒤

최종 평가
test로 고르면 test도 선택에 쓰인 셈
④ 측정·선택

☐ 실험 한 번 (조건마다 1회) ☐ Epoch 반복 ☐ Batch 한 번 (가장 자주 실행) ■ 모델 내부 (forward 안) ☐ 굵은 테두리 칩: 실험표의 핵심 손잡이와 판단 근거

SAME MAP, IN CODE

같은 지도를 코드로 보기

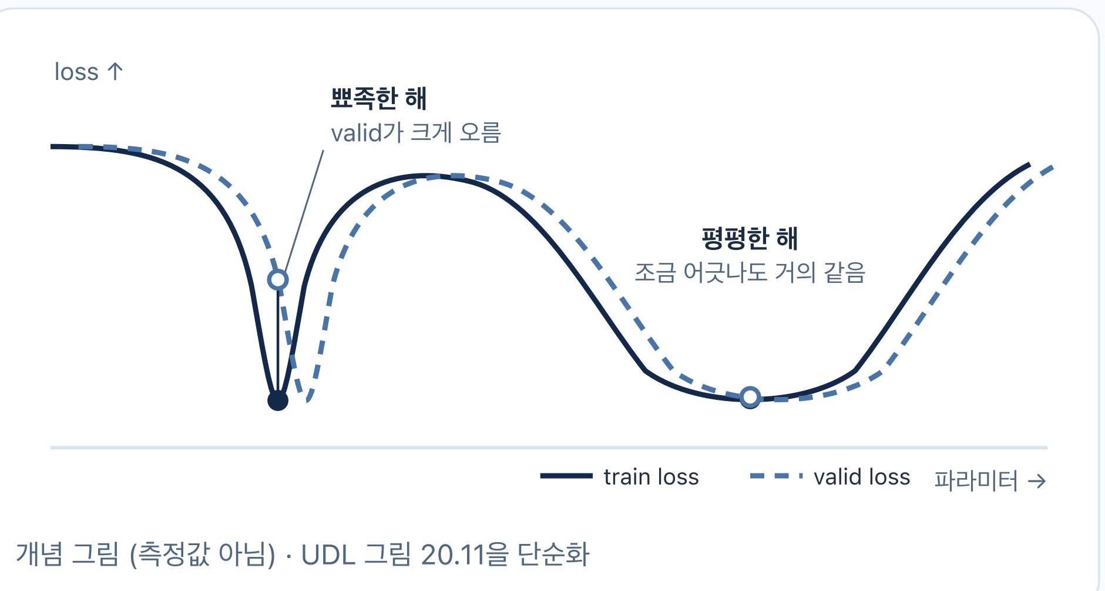
지도의 상자 하나 = 코드의 반복문 하나. 들여쓰기가 깊을수록 안쪽 상자이고 더 자주 실행됩니다.

Python · 설명용 골격 (실습 원문 아님)

```python
# ── 실험 한 번 ── 조건 하나마다 1회 ──────────────────────
torch.manual_seed(42)
train_loader = DataLoader(train_ds, batch_size=32, shuffle=True)
valid_loader = DataLoader(valid_ds, batch_size=256, shuffle=False)
model = ANN()
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.AdamW(model.parameters(),
                              lr=1e-3, weight_decay=0.01)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode="min", factor=0.5, patience=2)
best_loss, best_state, wait, patience = float("inf"), None, 0, 5

# ── Epoch 반복 ── epochs번 ─────────────────────────────
for epoch in range(epochs):
    model.train()

    # ── Batch 한 번 ── ⌈N ÷ batch_size⌉번 ──────────────
    for x, y in train_loader:
        optimizer.zero_grad()
        pred = model(x)
        loss = loss_fn(pred, y)
        loss.backward()
        optimizer.step()

    # ── Epoch 끝 ─────────────────────────────────────
    model.eval()
    with torch.no_grad():
        valid_loss = evaluate(model, valid_loader)
    scheduler.step(valid_loss)
    if valid_loss < best_loss:
        best_loss, wait = valid_loss, 0
        best_state = copy.deepcopy(model.state_dict())
    else:
        wait += 1
        if wait >= patience:
            break

# ── 학습이 끝난 뒤 ──────────────────────────────────
model.load_state_dict(best_state)
# 실험표에 기록 → 모든 선택이 끝난 뒤 test로 한 번 평가
```

GOOD SOLUTION

③ 어느 해에 도착하나 · 뾰족한 해와 평평한 해



개념 그림 (측정값 아님) · UDL 그림 20.11을 단순화

1. train loss만 보면 두 골짜기의 깊이는 같습니다.
2. valid의 손실 곡선은 train과 **조금 어긋나** 있어서, 뾰족한 골짜기에서는 valid loss가 크게 오릅니다.
3. 지도에서 ③이 붙은 도구들은 **평평한 쪽**으로 기울게 만드는 장치로 볼 수 있습니다.

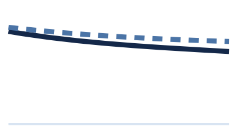
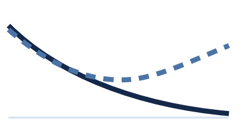
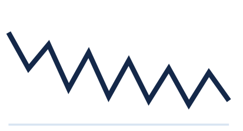
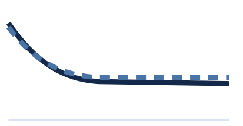
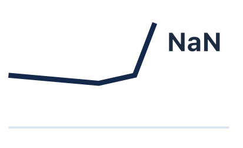
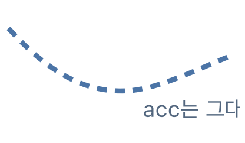
대표적인 해석이며, 일반화를 완전히 설명하는 이론은 아직 없습니다.

DIAGNOSIS

증상에서 도구로 · 학습곡선을 보고 지도에서 찾기

학습곡선의 모양이 "어느 목표가 문제인지"를 알려 주고, 지도가 "어느 상자에서 고칠지"를 알려 줍니다. 처방은 강의 PDF p.82 기준입니다.

실선은 train loss, 점선은 valid loss의 모양 예시 (측정값 아님)

곡선 모양	관찰	어느 목표의 문제	지도에서 고칠 곳	먼저 바꿀 것
	**train·valid 둘 다 나쁨**	② 최적화 부족 또는 모델 용량 부족 (Underfitting)	갱신: lr Epoch 수 모델 용량	lr·epoch·모델 용량 늘리기. 데이터·loss·기울기 확인
	**train 좋아짐 · valid 나빠짐**	③ 일반화 (Overfitting)	Early Stopping Dropout Weight Decay	Early Stopping · Dropout · Weight Decay. 데이터 양·라벨 잡음 확인
	**loss가 크게 진동**	② 최적화 불안정 (보폭 과대 · 기울기 잡음 과대)	갱신: lr Batch size	lr 줄이기 · batch 늘리기. 입력 정규화 확인
	**초반 개선 후 정체**	② 최적화 (plateau)	Scheduler Optimizer	Scheduler 적용 · Optimizer 후보 비교. 기울기 크기·활성화 확인
	**loss가 NaN/Inf**	② 수치 불안정	갱신: lr Loss · 입력	lr 줄이기 · 입력·loss 검사. 필요하면 gradient clipping·dtype
	**valid acc는 그대로인데 valid loss만 오름**	① 손실의 성질: 틀린 예측을 점점 더 확신	Early Stopping 감시 지표	무엇을 기준으로 best를 고를지 정하기 (UDL §8.1 보충)

곡선을 보고 먼저 할 질문 (PDF p.86): train과 valid 중 어느 쪽이 문제인가? 초반부터인가, 후반부터인가? loss와 accuracy가 같은 결론인가? 한 설정만 바꾸었나?

새싹 NVIDIA AI 코어 엔지니어 2기 · Day10 학습 노트 · 2026.10.30